Carbonly — Technical Architecture

Version: 1.0 **Last updated:** 2026-04-15 **Status:** Active development

Carbonly is a multi-tenant SaaS platform for tracking, reporting, and reducing corporate greenhouse gas (GHG) emissions in accordance with the **GHG Protocol** (Scopes 1, 2 & 3) and **ISO 14064**. This document describes the end-to-end technical architecture — from the landing page to the Stripe billing integration.

Table of Contents

1. System Overview
2. Technology Stack
3. High-Level Architecture
4. Database Design
5. Multi-Tenancy Model
6. Authentication & Authorization
7. Application Structure
8. Calculation Engine
9. Reporting Pipeline
10. Billing & Stripe Integration
11. API Surface
12. Data Flows
13. Security Model
14. Deployment Topology
15. Operational Concerns

---

1. System Overview

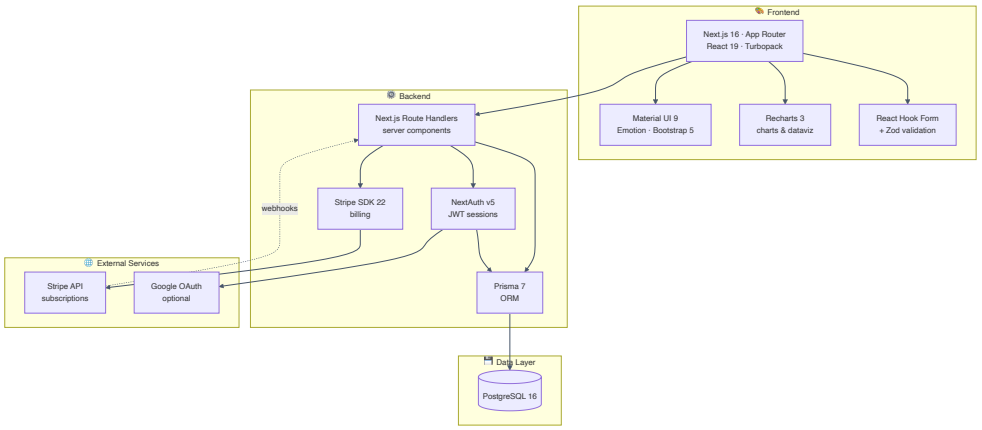
Carbonly is a **server-rendered React application** that turns raw activity data (fuel burned, kWh consumed, business travel, etc.) into audit-ready CO_2e reports. Every organisation ("entity") is an isolated tenant with its own emission records,

yearly configurations, and reports, but shares a **global taxonomy** of scopes, activities, and emission factors maintained by platform admins.

Key capabilities

CAPABILITY	DESCRIPTION
Multi-tenant dashboard	Each entity sees its own real-time emissions by scope, activity, month, year
GHG Protocol calculations	Scope 1 (direct), Scope 2 (purchased energy), Scope 3 (value chain) — all via `quantity × emissionFactor`
Annual reporting	One-click generation of ISO 14064-aligned reports with CSV export
Subscription billing	Stripe-backed tiered plans (Starter / Professional / Enterprise) with 14-day trial
Super-admin console	Platform-wide management of tenants, users, taxonomy, and billing analytics
Role-based access	`SUPER_ADMIN`, `ADMIN`, `EXPERT` at the platform level; `ADMIN`, `EXPERT` per entity

2. Technology Stack



Dependency inventory (production)

CATEGORY	LIBRARY	VERSION	ROLE
Framework	`next`	16.2.3	App Router, server components, SSR
	`react` / `react-dom`	19.2.4	UI runtime
UI	`@mui/material`	^9.0.0	Component library
	`@mui/material-nextjs`	^9.0.0	App Router SSR cache
	`@mui/icons-material`	^9.0.0	Material icons
	`@emotion/react`, `@emotion/styled`, `@emotion/cache`	^11.14	MUI styling engine
	`bootstrap`	^5.3.8	Responsive grid utilities
	`lucide-react`	^1.8.0	Icon set (non-Material)
	`recharts`	^3.8.1	Charts (pie, area, bar)
Forms	`react-hook-form`	^7.72.1	Form state management
	`@hookform/resolvers`	^5.2.2	Validation bridge
	`zod`	^4.3.6	Schema validation
Auth	`next-auth`	^5.0.0-beta.30	Authentication
	`@auth/prisma-adapter`	^2.11.1	Session + user persistence
	`bcryptjs`	^3.0.3	Password hashing
Database	`prisma` / `@prisma/client`	^7.7.0	Query builder + migrations
	`@prisma/adapter-pg`	^7.7.0	Node.js runtime adapter
	`pg`	^8.20.0	PostgreSQL driver
Billing	`stripe`	^22.0.1	Stripe server SDK

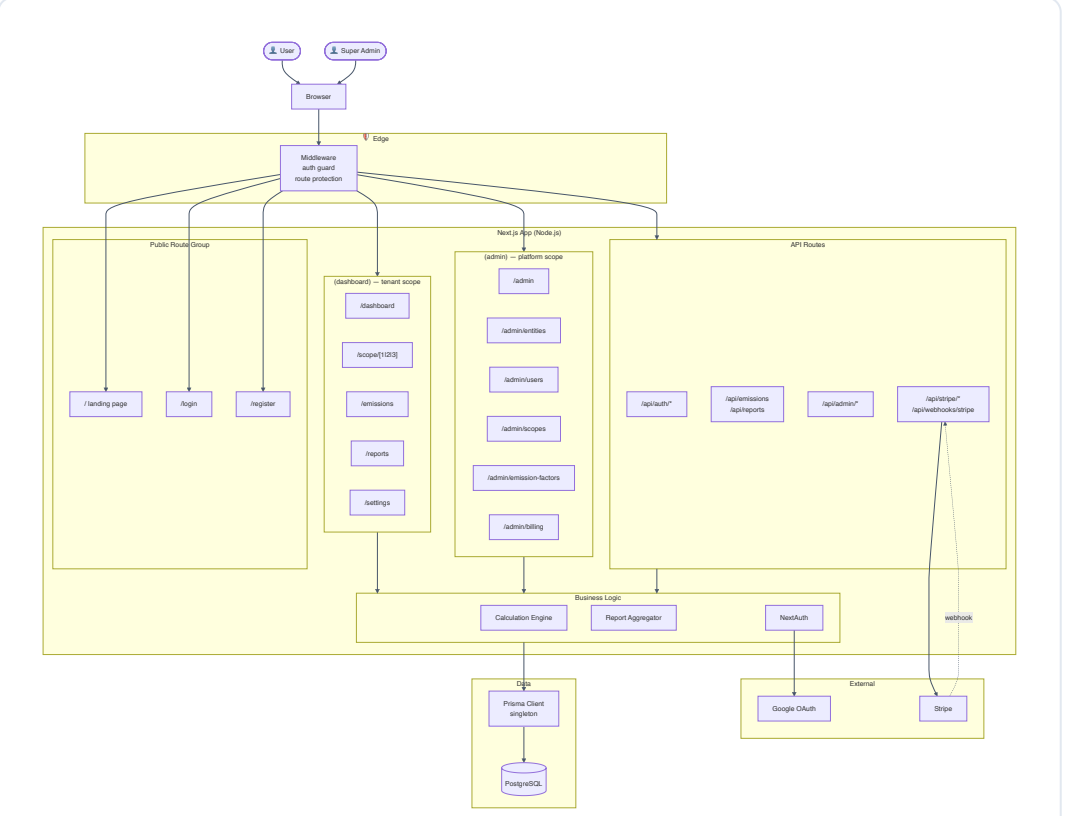
CATEGORY	LIBRARY	VERSION	ROLE
	`@stripe/stripe-js`	^9.1.0	Stripe.js (checkout redirects)
State	`zustand`	^5.0.12	Client-side state (reserved)
Utilities	`date-fns`	^4.1.0	Date formatting

Runtime & language

- **Node.js** ≥ 18 (LTS recommended)
- **TypeScript 5.9** — strict mode
- **ESLint 9** — Next.js shareable config

---

3. High-Level Architecture



Request lifecycle

1. **Browser request** → Edge middleware (NextAuth JWT cookie check)

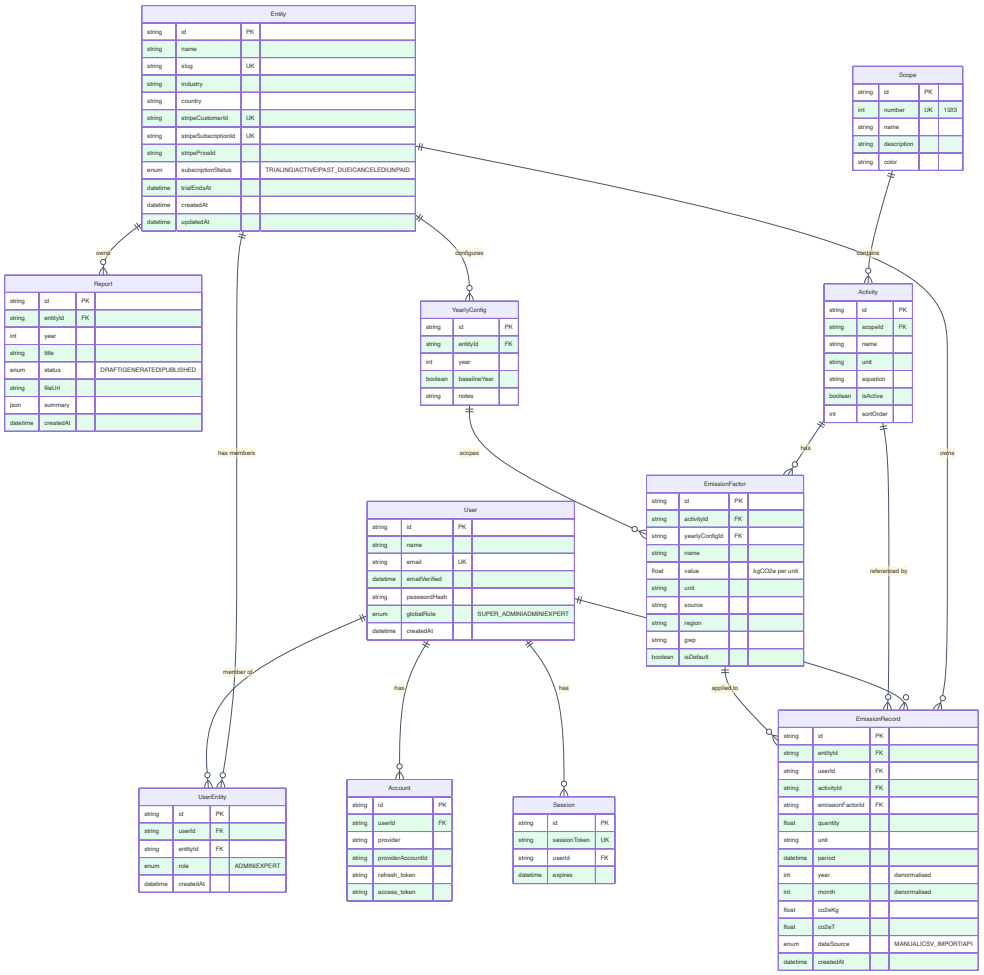
2. **Public paths** (`/login`, `/register`, `/api/auth/*`, `/api/webhooks/*`) → bypass auth
3. **Authenticated paths** → session is decoded; `user.globalRole`, `user.entityId`, `user.entitySlug` are available on every request
4. **Server components** render on the Node.js runtime (not Edge) so Prisma + bcrypt work
5. **Client components** (marked `"use client"`) hydrate after initial SSR
6. **Data mutations** go through route handlers (`/api/*`) that validate the session and write through the Prisma singleton

---

4. Database Design

Carbonly uses **PostgreSQL 16** with **Prisma 7** as the ORM. The schema is organised into five domains: multi-tenancy, authentication, taxonomy, emissions, and reports.

Entity-relationship diagram



Schema highlights

Domains:

DOMAIN	TABLES	PURPOSE
Tenancy	`Entity`, `UserEntity`	Organisation and membership
Auth	`User`, `Account`, `Session`, `VerificationToken`	NextAuth + Prisma adapter
Config	`YearlyConfig`	Per-entity, per-year settings (baseline year flag)
Taxonomy	`Scope`, `Activity`, `EmissionFactor`	Shared global catalogue
Data	`EmissionRecord`	Per-tenant activity data
Reports	`Report`	Aggregated annual outputs

Key design decisions:

1. **Denormalised `year` and `month` on `EmissionRecord`** — avoids `EXTRACT()` in every dashboard query; powers the composite index `[entityId, year]`
2. **Pre-computed `co2eKg` and `co2eT`** — the calculation is done at write time, so reads are instant
3. **Global taxonomy (`Scope` / `Activity` / `EmissionFactor`)** — all tenants share the same scopes and factors to ensure auditability, but can override via `YearlyConfig`-scoped factors
4. **Cascade deletes** — removing an `Entity` deletes its users (`UserEntity`), emission records, and reports
5. **`Report.summary` JSON** — flexible schema-less blob storing aggregated scope breakdowns, monthly trends, and top activities at generation time (immutable snapshot)

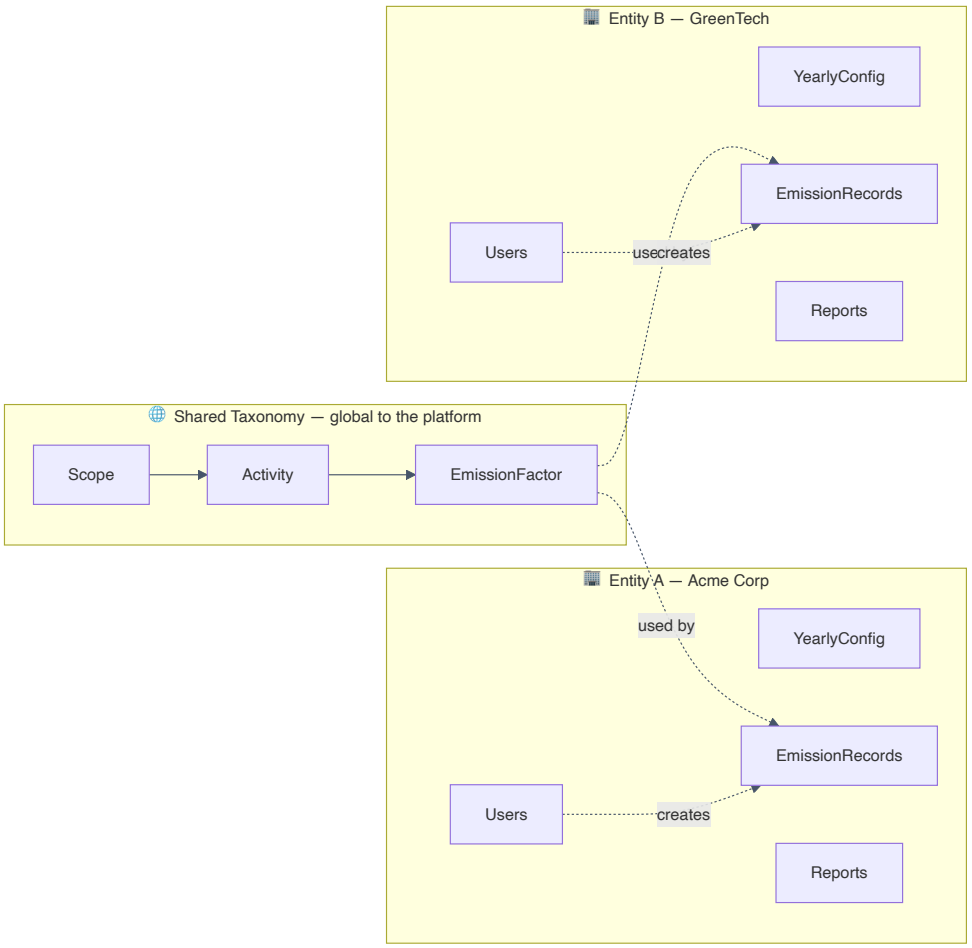
Indexes

TABLE	INDEX	RATIONALE
`Entity`	`slug` (unique)	URL lookups
`Entity`	`stripeCustomerId`, `stripeSubscriptionId` (unique)	Webhook routing
`User`	`email` (unique)	Credentials lookup
`UserEntity`	`(userId, entityId)` (unique)	Prevent duplicate membership
`Activity`	`scopeId`	Scope → activities query
`EmissionFactor`	`activityId`	Activity → factors query
`EmissionRecord`	`(entityId, year)`	Dashboard data-fetch
`EmissionRecord`	`(entityId, activityId)`	Activity breakdown

---

5. Multi-Tenancy Model

Carbonly is **shared-database, shared-schema multi-tenant**: every tenant-scoped table carries an `entityId` foreign key, and every query filters on it.



Access rules

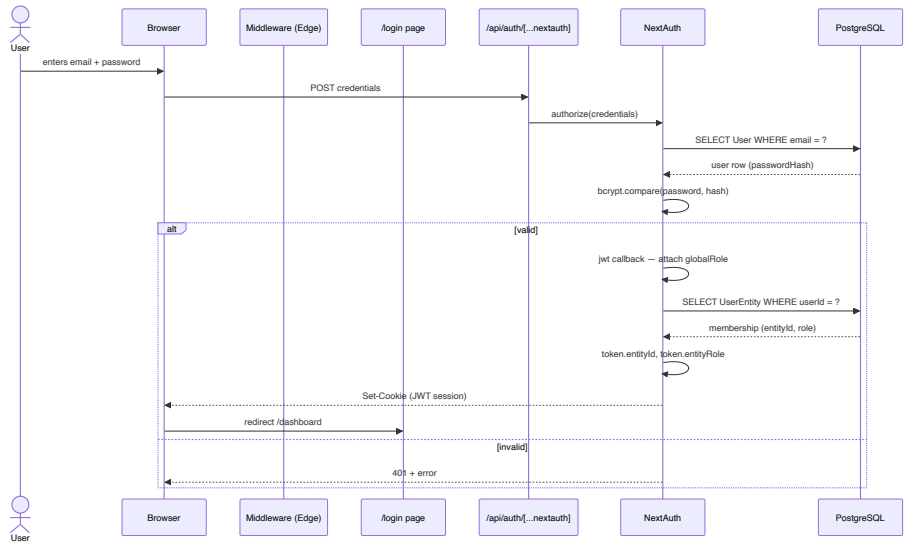
PATH	WHO	GUARD
`/dashboard`, `/emissions`, `/reports`, `/settings`, `/scope/*`	Any member of an entity	Session `entityId` present
`/admin/*`	Any authenticated user (demo mode)	Session `user` present
`POST /api/emissions`	Any member	Record is created with `entityId` from session
`POST /api/admin/*`	Any authenticated user (demo mode)	—

Every query that touches tenant data has `where: { entityId: session.user.entityId }` — enforced in each page/route handler. This is the single most important rule for isolation.

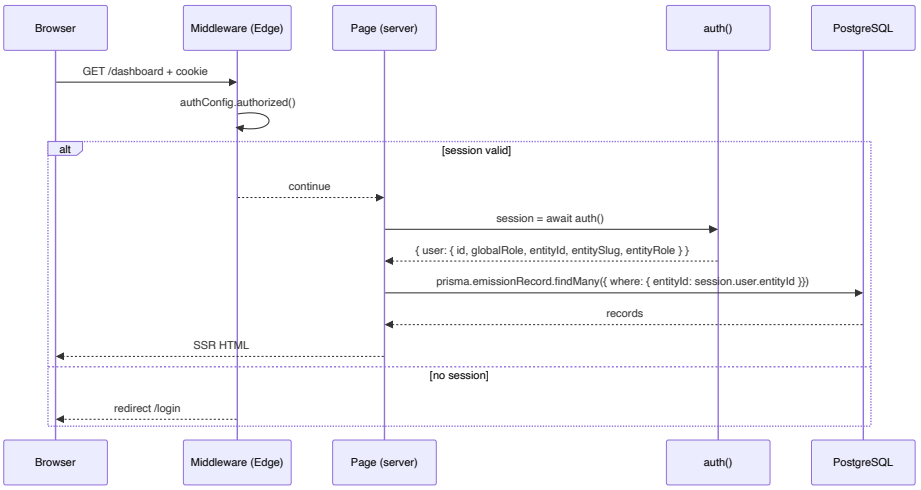
6. Authentication & Authorization

Carbonly uses **NextAuth v5 (Auth.js)** with a **JWT session strategy**. Sessions are stored as signed cookies, not in the database, which means every server component can synchronously read `session.user` without a round-trip.

Sign-in sequence (credentials)



Request authorization



Security layers

1. **Edge middleware** (`src/middleware.ts`) — rejects unauthenticated requests at the edge using the lightweight `authConfig` (no Prisma/Node.js imports allowed in Edge runtime)
2. **Full Auth config** (`src/lib/auth/auth.ts`) — used by server components and API routes for full session decoding via `PrismaAdapter`
3. **Per-route checks** — every mutation route re-validates `session.user` before acting
4. **Tenant filter** — all Prisma queries filter by `entityId` from session

Demo mode note

For portfolio/demo purposes, the `/admin` route is accessible to any authenticated user. Production would re-enable the `SUPER_ADMIN` check in `authConfig.authorized()` and in the admin layout.

---

7. Application Structure

Next.js 16 **App Router** with three route groups:

```
src/app/
├── layout.tsx                    # Root layout — Bootstrap CSS + MUI Th
├── globals.css                   # Keyframes + utility CSS compatibilit
├── page.tsx                      # Landing page (public)
│
├── (auth)/                       # Route group: minimal layout, green g
│   ├── layout.tsx
│   ├── login/page.tsx
│   └── register/page.tsx
│
├── (dashboard)/                  # Route group: sidebar + topbar
│   ├── layout.tsx                # MUI Drawer (permanent on ≥lg, tempor
│   ├── dashboard/page.tsx        # Main tenant dashboard
│   ├── scope/[scopeNumber]/      # Scope 1/2/3 calculators
│   ├── emissions/page.tsx        # All records
│   ├── reports/page.tsx          # Report list + detail
│   └── settings/page.tsx         # Profile, org, billing
│
├── (admin)/                      # Route group: dark platform UI
│   ├── layout.tsx                # MUI Drawer with dark theme
│   └── admin/
│       ├── page.tsx              # Platform overview
│       ├── entities/page.tsx
│       ├── users/page.tsx
│       ├── scopes/page.tsx
│       ├── emission-factors/page.tsx
│       └── billing/page.tsx
│
└── api/                          # Route handlers (JSON)
    ├── auth/[...nextauth]/       # NextAuth
    ├── auth/register/            # Sign-up
    ├── emissions/                # GET, POST
    ├── reports/                  # GET, POST
    ├── reports/[reportId]/       # PATCH, DELETE
    ├── reports/[reportId]/export # CSV export
    ├── admin/entities/           # POST + PATCH, DELETE by ID
    ├── admin/users/              # POST + PATCH, DELETE by ID
    ├── admin/activities/         # POST + PATCH, DELETE by ID
    ├── admin/emission-factors/   # POST + PATCH, DELETE by ID
    ├── stripe/checkout/          # POST
    ├── stripe/portal/            # POST
    └── webhooks/stripe/          # POST (webhook)
```

Component organisation

```
src/components/
├── admin/              # Client CRUD components for admin panel
├── auth/               # Login + register forms
├── calculator/         # Scope calculator (new emission record form)
├── dashboard/          # KPI cards, charts, tables
├── landing/            # Marketing page building blocks
├── layout/             # Sidebar, topbar, admin sidebar, role switcher
├── providers/          # ThemeProvider (MUI + Emotion + SessionProvider
├── reports/            # Report list, detail, generate drawer
└── settings/           # Billing section
```

Server vs. client components

- **Server components by default** — all pages are server components unless explicitly marked
- `"use client"` **used only when needed** — interactivity, form state, hooks, event handlers, MUI Drawer state
- **Data fetching** — always in server components via `prisma` directly (no `/api/*` round-trip for SSR)
- **Mutations** — always go through `/api/*` route handlers (client → server)

---

8. Calculation Engine

All emissions math flows through a **pure functional engine** in `src/lib/calculator/engine.ts`.

Formula

```
CO₂e (kg) = quantity × emissionFactor.value
CO₂e (t)  = CO₂e (kg) / 1000
```

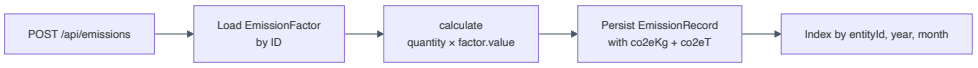
Engine API

```
interface CalculationInput {
  quantity: number;            // e.g. 12,400
  emissionFactorValue: number; // kgCO₂e per unit, e.g. 0.20493 for UK g
}

interface CalculationResult {
  co2eKg: number;  // rounded to 3 decimal places
  co2eT: number;   // rounded to 6 decimal places
}

calculate(input): CalculationResult
calculateScope1(input): CalculationResult  // delegates
calculateScope2(input): CalculationResult  // delegates
calculateScope3(input): CalculationResult  // delegates
aggregateTotals(records): { totalCo2eKg, totalCo2eT }
```

Call site

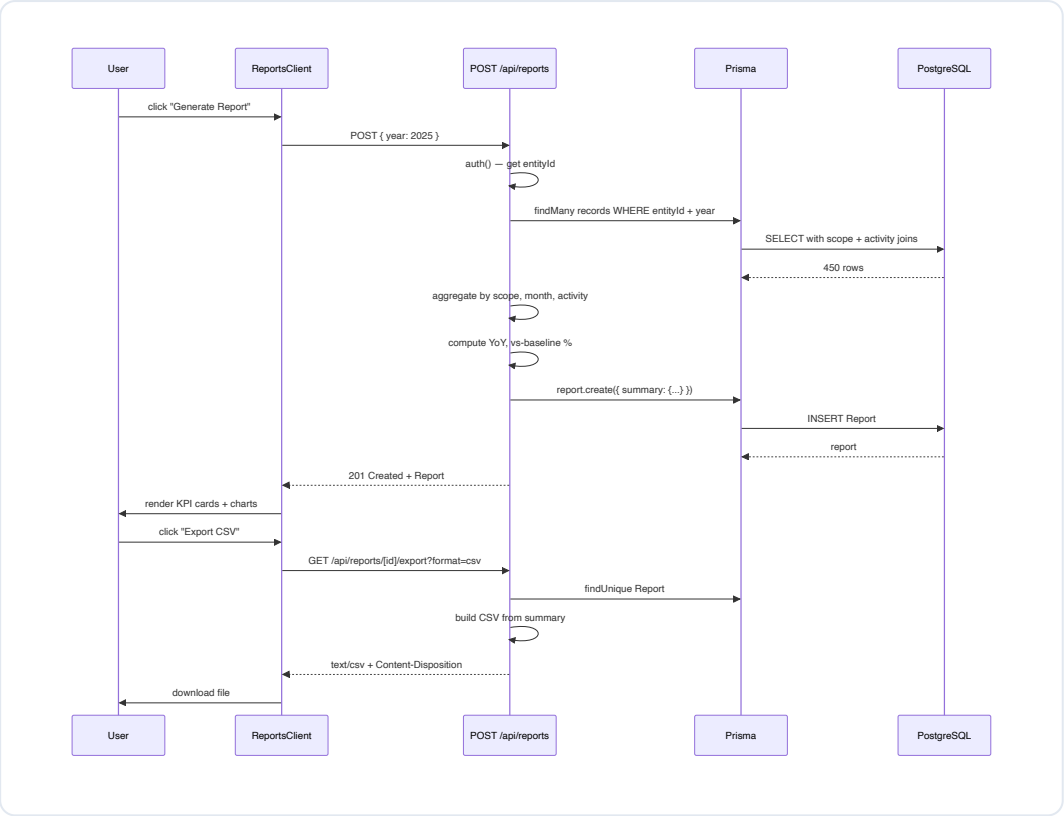


All CO_2e values are **written once at create-time** — no on-read recalculation. This keeps the dashboard aggregations sub-10ms.

---

9. Reporting Pipeline

Reports are **immutable snapshots** of a tenant's annual emissions, with all aggregations stored in a JSON blob at generation time.



Report summary shape

```
{
  "totalCo2eT": 1248.5,
  "recordCount": 450,
  "scope1": { "co2eT": 342.1, "pct": 27.4, "activities": [...] },
  "scope2": { "co2eT": 518.4, "pct": 41.5, "activities": [...] },
  "scope3": { "co2eT": 388.0, "pct": 31.1, "activities": [...] },
  "byMonth": [{ "month": 1, "co2eT": 92.3 }, ...],
  "topActivities": [{ "name": "Grid electricity", "co2eT": 234.1, "scope
  "baselineCo2eT": 1812.4,
  "prevYearCo2eT": 1418.0,
  "yoyChangePct": -11.95,
  "vsBaselinePct": -31.12,
  "entityName": "Acme Corp",
  "generatedAt": "2026-04-15T10:23:00Z"
}
```

Lifecycle

```
DRAFT  →  GENERATED  →  PUBLISHED
```

- **DRAFT** — legacy state (not written by current code)
- **GENERATED** — aggregation complete, downloadable as CSV
- **PUBLISHED** — marked ready for stakeholders (toggleable from UI)

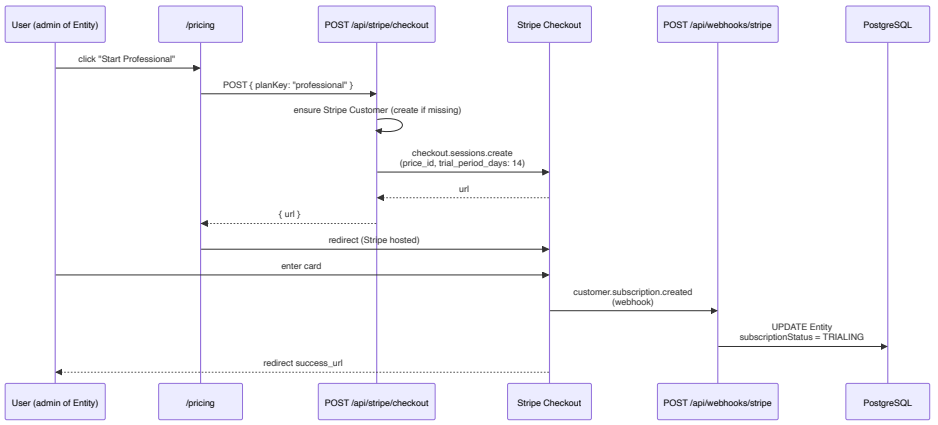
10. Billing & Stripe Integration

Carbonly uses Stripe Subscriptions with three tiers. Subscription state is mirrored into the `Entity` table via webhooks.

Plans

PLAN	MONTHLY (£)	ANNUAL (£/MO)	TRIAL	USERS	SCOPES	HISTORY
Starter	49	39	14 days	3	1, 2	1 year
Professional	149	119	14 days	15	1, 2, 3	unlimited
Enterprise	custom	custom	—	unlimited	1, 2, 3	unlimited

Checkout → activation



Webhook events handled

EVENT	ACTION
`customer.subscription.created`	Store `stripeSubscriptionId`, `stripePriceId`, status
`customer.subscription.updated`	Update price, plan, or status
`customer.subscription.deleted`	Set status → `CANCELED`
`invoice.payment_failed`	Set status → `PAST_DUE`

All webhooks are signature-verified with `STRIPE_WEBHOOK_SECRET` — unverified payloads are rejected 400.

Stripe ↔ Carbonly mapping

STRIPE STATUS	`SUBSCRIPTIONSTATUS`
`trialing`	TRIALING
`active`	ACTIVE
`past_due`	PAST_DUE
`canceled`	CANCELED
`unpaid`	UNPAID
`incomplete`, `paused`	PAST_DUE (fallback: CANCELED)

Customer portal

Users click **"Manage billing"** in settings → `POST /api/stripe/portal` → server creates a billing portal session with `stripeCustomerId` → client redirects to the Stripe-hosted portal for card updates, invoice history, plan changes, and cancellations.

---

11. API Surface

All routes are Next.js **Route Handlers** returning JSON (except CSV export and webhooks).

Public / auth

METHOD	PATH	PURPOSE
GET, POST	`/api/auth/[...nextauth]`	NextAuth — sign in, sign out, callbacks, CSRF
POST	`/api/auth/register`	Create user + hash password

Tenant-scoped (session required)

METHOD	PATH	PURPOSE
GET	`/api/emissions`	List records for `entityId` and `year` (query param)
POST	`/api/emissions`	Create record; auto-calculate CO_2e
GET	`/api/reports`	List reports for `entityId`
POST	`/api/reports`	Generate a new report for a given year
PATCH	`/api/reports/[reportId]`	Update status
DELETE	`/api/reports/[reportId]`	Remove
GET	`/api/reports/[reportId]/export`	Stream CSV download

Admin-scoped (any session in demo mode)

METHOD	PATH	PURPOSE
POST / PATCH / DELETE	`/api/admin/entities[/:id]`	CRUD tenants
POST / PATCH / DELETE	`/api/admin/users[/:id]`	CRUD users
POST / PATCH / DELETE	`/api/admin/activities[/:id]`	CRUD activities
POST / PATCH / DELETE	`/api/admin/emission-factors[/:id]`	CRUD factors

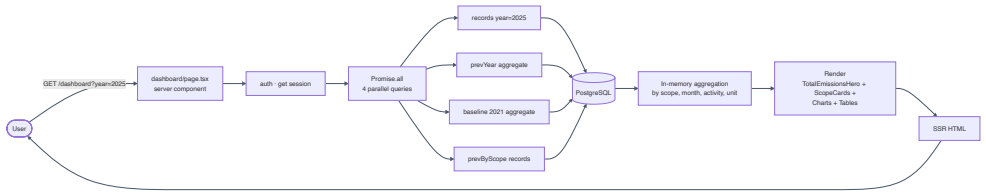
Billing

METHOD	PATH	PURPOSE
POST	`/api/stripe/checkout`	Create hosted checkout session
POST	`/api/stripe/portal`	Create customer portal session
POST	`/api/webhooks/stripe`	Handle subscription lifecycle events

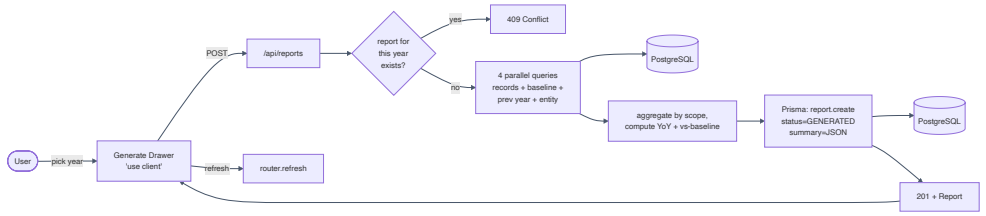
12. Data Flows

Emission record creation

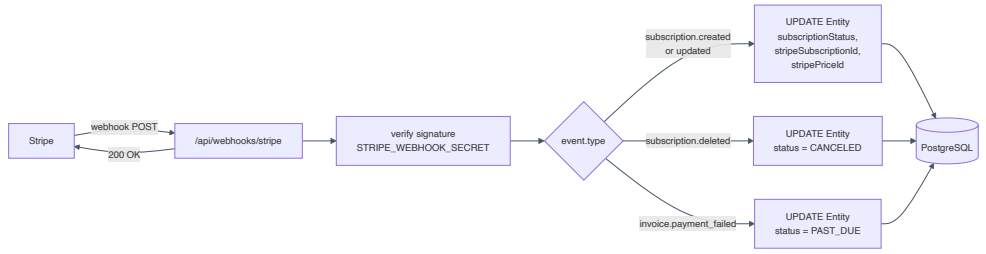
Dashboard read



Report generation



Stripe subscription update



13. Security Model

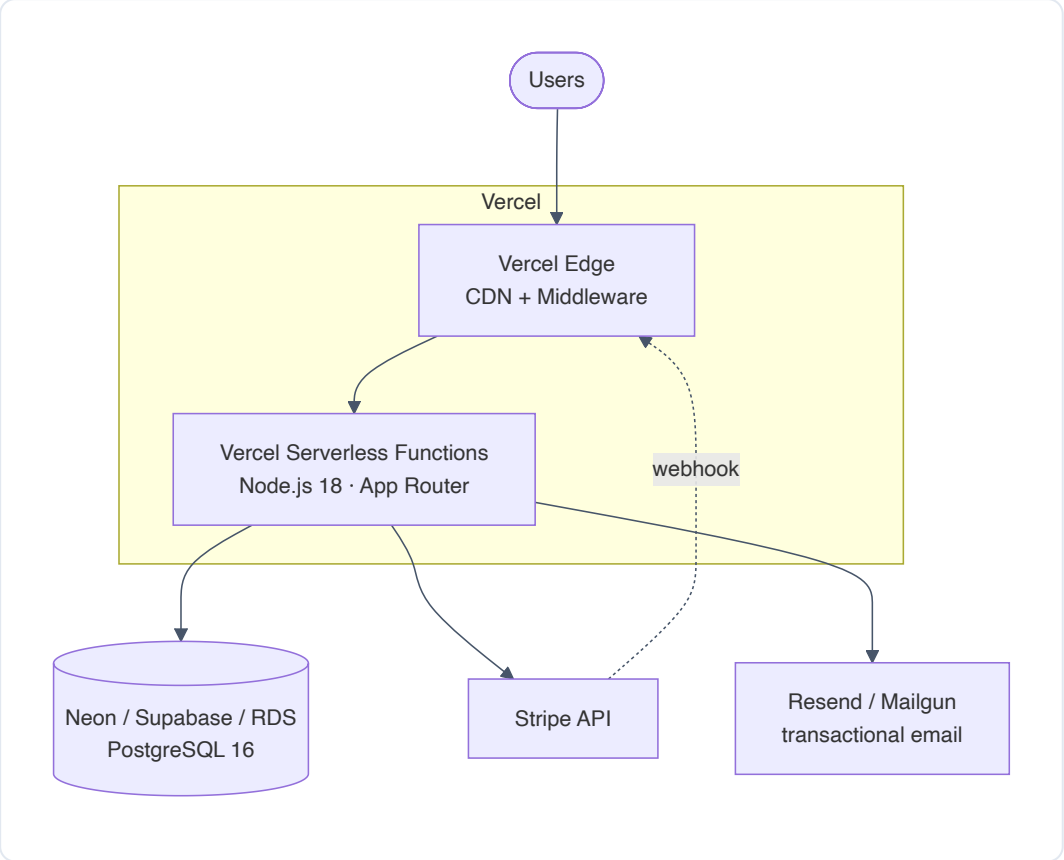
THREAT	MITIGATION
Credential theft	Passwords hashed with `bcryptjs` (cost factor 10 in seed; configurable)
Session hijacking	HttpOnly, SameSite=Lax cookies signed with `NEXTAUTH_SECRET`
CSRF	NextAuth built-in CSRF tokens for sign-in/sign-out; mutation routes require valid session cookie
Cross-tenant access	Every Prisma query filters on `entityId` from the session — never trust client-sent `entityId`
Stripe webhook forgery	`stripe.webhooks.constructEvent()` with signing secret — rejects unsigned bodies
Prisma injection	ORM parameterises all queries; no raw SQL in application code
XSS	React escapes by default; MUI components are safe; `dangerouslySetInnerHTML` not used
Secrets	All sensitive config in `.env` (excluded from git); server-only access
Middleware allowlist	`/api/webhooks/*` is the only unauthenticated POST endpoint

Environment variables

VARIABLE	REQUIRED	PURPOSE
`DATABASE_URL`	✅	PostgreSQL connection string
`NEXTAUTH_URL`	✅	Canonical URL (e.g. `http://localhost:3000`)
`NEXTAUTH_SECRET`	✅	JWT signing key (32+ random bytes)
`GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET`	⚪	Enable Google OAuth
`STRIPE_SECRET_KEY`	⚪	Enable billing
`STRIPE_WEBHOOK_SECRET`	⚪	Verify webhooks
`STRIPE_PRICE_STARTER` / `_PROFESSIONAL` / `_ENTERPRISE`	⚪	Plan → Stripe price ID

14. Deployment Topology

Recommended: Vercel + managed Postgres



Vercel-specific notes

- **Edge Runtime** for middleware (NextAuth edge-safe config)
- **Node.js Runtime** for everything else (Prisma requires Node)
- **Serverless functions** auto-scale; Prisma client uses connection pooling via the serverless adapter
- **Webhook URL** registered in Stripe dashboard: `https://<app>.vercel.app/api/webhooks/stripe`

Local development

- Postgres via Homebrew (`brew install postgresql@16`)
- `DATABASE_URL="postgres://postgres:postgres@localhost:5432/carbonly"`
- `npm run dev` → Turbopack dev server on port 3000
- `npm run db:migrate` → Prisma Migrate
- `npm run db:seed` → demo users + taxonomy
- `npm run db:seed-records` → ~1,900 demo emission records

15. Operational Concerns

Performance budgets

METRIC	TARGET	CURRENT (DEV)
First contentful paint (dashboard)	< 1.5s	~200ms on localhost
Dashboard data query	< 50ms	20–35ms with 1,900 records
Record creation	< 100ms	~30ms
Report generation	< 500ms	~120ms for 450 records

Monitoring (when deployed)

- **Application logs** — Vercel Runtime Logs (structured JSON)
- **Prisma query logs** — `error` level in production, `query` in dev
- **Stripe webhook delivery** — dashboard retry dashboard
- **Error tracking** — integrate Sentry for client + server errors

Known limitations

1. **No email sending** — `VerificationToken` table exists but no SMTP provider is wired; register flow doesn't verify emails
2. **Report PDF generation** — CSV export works; PDF is not yet implemented
3. **No tests** — unit/integration test suite not yet established
4. **Admin role check relaxed for demo** — production should re-enable `SUPER_ADMIN` guard
5. **No audit log** — platform actions are not logged for forensic purposes

Roadmap items

- Stripe billing portal integration with usage metering (per-record limits)
- Email verification + password reset flows
- PDF export via Puppeteer or `@react-pdf/renderer`
- Tenant-scoped API keys for CSV import from third-party tools
- SAML SSO for Enterprise tier
- SOC 2 audit trail (append-only log table)

Appendix A — File structure reference

```
Carbonly/
├── prisma/
│   ├── schema.prisma          # Source of truth for DB schema
│   ├── migrations/            # Generated migration SQL
│   ├── seed.ts                # Taxonomy + demo users
│   └── seed-records.ts        # ~1,900 demo emission records
│
├── src/
│   ├── app/                   # Next.js App Router (see §7)
│   ├── components/            # React components (see §7)
│   ├── lib/
│   │   ├── auth/              # NextAuth config (edge + full)
│   │   ├── db/prisma.ts       # Prisma client singleton
│   │   ├── calculator/        # Pure calculation engine
│   │   ├── stripe/plans.ts    # Plan catalogue
│   │   ├── stripe/client.ts   # Stripe SDK singleton
│   │   ├── validations/       # Zod schemas
│   │   └── theme.ts           # MUI theme (green palette)
│   ├── generated/prisma/      # Prisma-generated client (gitignored)
│   ├── types/                 # NextAuth module augmentation
│   └── middleware.ts          # Edge auth guard
│
├── docs/
│   └── ARCHITECTURE.md        # This document
│
├── .env                       # Secrets (gitignored)
├── package.json
├── tsconfig.json
└── next.config.ts
```

Appendix B — Glossary

TERM	MEANING
Entity	A tenant organisation (e.g. "Acme Corp")
Scope 1	Direct emissions from owned/controlled sources
Scope 2	Indirect emissions from purchased energy
Scope 3	All other indirect emissions in the value chain
Emission Factor	Coefficient converting activity data → CO_2e (e.g. 0.20493 $kgCO_2e/kWh$ for UK grid)
CO_2e	Carbon dioxide equivalent — unified unit for all GHGs
GWP	Global Warming Potential — weighting for non-CO_2 gases (e.g. CH_4 = 28× CO_2 over 100 years)
GHG Protocol	Corporate Accounting Standard by WRI/WBCSD
ISO 14064	International standard for GHG inventory & verification
Baseline year	Reference year against which reduction targets are measured